CodeSport

Sandya Mannarswamy

Welcome to CodeSport. This month, we feature a medley of questions about operating systems, computer architecture and algorithms.

Last month's column featured three questions on mutual exclusion, the memory consistency model and synchronisation. Since none of the replies I received were entirely correct, we will keep the questions open this month too. Since there were requests from readers for another set of interview questions, we decided to postpone our discussion of memory fences to next month, and instead feature a medley of questions regarding operating systems, computer architecture and algorithms.

OS-related questions

1. We know that the set of memory locations that a process is allowed to access is called the address space of the process. Consider a program that reads an array of 1,000,000 integers. Can this program be executed as a process on a machine whose physical memory is only 400,000 bytes?
2. We know that operating systems are typically multi-programmed—which means that multiple processes can be active at the same time. Can there be multiple kernels executing at the same time?
3. Is it possible to support multiple processes without support for virtual memory?
4. Consider the following code snippet. What signal gets delivered to the process when this code segment is executed?

```
int main(void)
{
    int* array = (int*) malloc(10);
    array++;
   *array = 128000;
    printf("value is %d\n", *array);
}
```

5. Consider the following code snippet, which shows how a process invokes the *fork()* and *exec()* system calls. Will it output the line *"execve succeeded"*? If not, why?

```
int result = fork();
if (result == 0) /* child code */
{
     if (execve("my new program", …) < 0)
     {
          printf("execve failed\n");
     }
     else
          printf("execve succeeded\n");
}
```

6. If a process is multi-threaded, and one of the threads invokes the *fork()* system call, what happens?
7. If a process is multi-threaded, and one of the threads invokes the *exec()* system call, what happens?
8. What is 'copy on write', and how is it used to reduce the *fork-exec* overhead?
9. Assume that a process detects and handles stack overflow by protecting its stack region boundary with a write-protected page. Hence, any write to this page due to stack overflow will result in a SIGSEGV being generated. Since the process stack has already overflowed, how can the signal handler for the SIGSEGV be executed?
10. Should threads be supported at the user level, the kernel level, or both? Justify your answer.

Computer architecture questions

1. What is a write-through cache, and what is a write-back cache? What are their advantages and disadvantages?
2. What are the advantages and disadvantages of associative caches over direct-mapped caches?
3. Why is the first-level cache in a processor typically direct-mapped, whereas the second and third levels are set-associative?
4. What is the difference between a little-endian architecture and a big-endian architecture?
5. What is the difference between a symmetric multiprocessor (SMP) system and a chip multiprocessor system?
6. What is the difference between write-allocate and write-no-allocate cache?
7. What is a victim cache?
8. What is the difference between simultaneous multi-threading, and switch-on-event multi-threading? What kind of threading is employed in Nehalem processors, and what is employed in Itanium processors?
9. Assume that you are a chip designer, and that you are given the choice of deciding on the number of registers the processor should have. If you are allowed to increase the number of registers, would you go on doing so indefinitely?